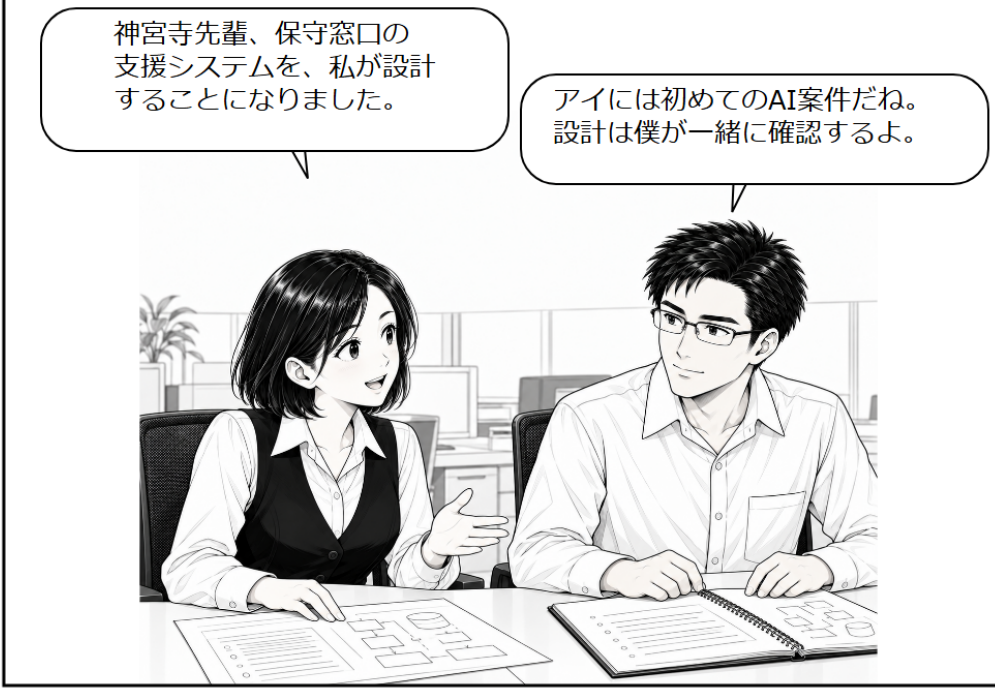
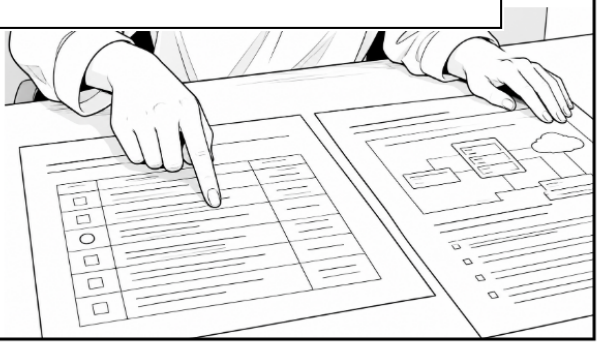
はじめに ― 荒川裕二の技術記事から



想定する問い合わせ

DEV001のサーバーで管理画面が開きません。どうしたらいいですか？

現地保守も頼めますか？



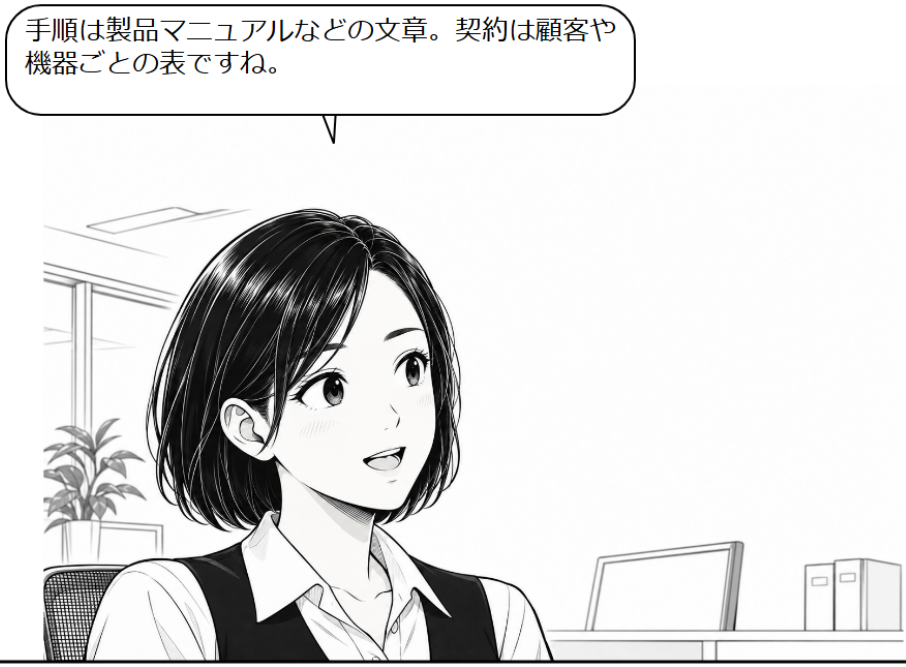
現状の確認や操作の手順、それに、訪問できる条件の確認が必要ですね。

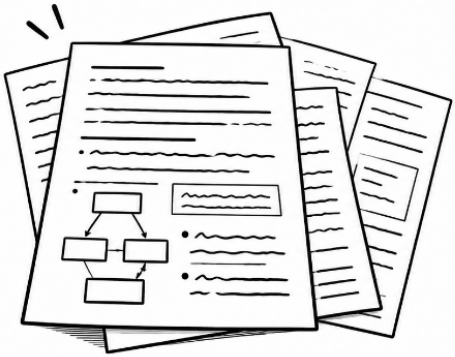
社内の最新手順と、この機器の契約が必要だね。

問い合わせへの対応案

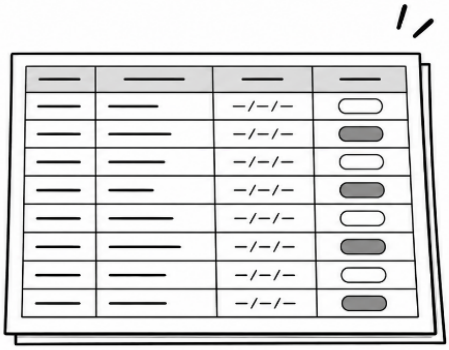
アイ
根拠のある確認手順と、この機器の契約条件を一緒に答えたいです。

1. 文書の意味検索と業務データの照会を組み合わせる





手順書：非構造化



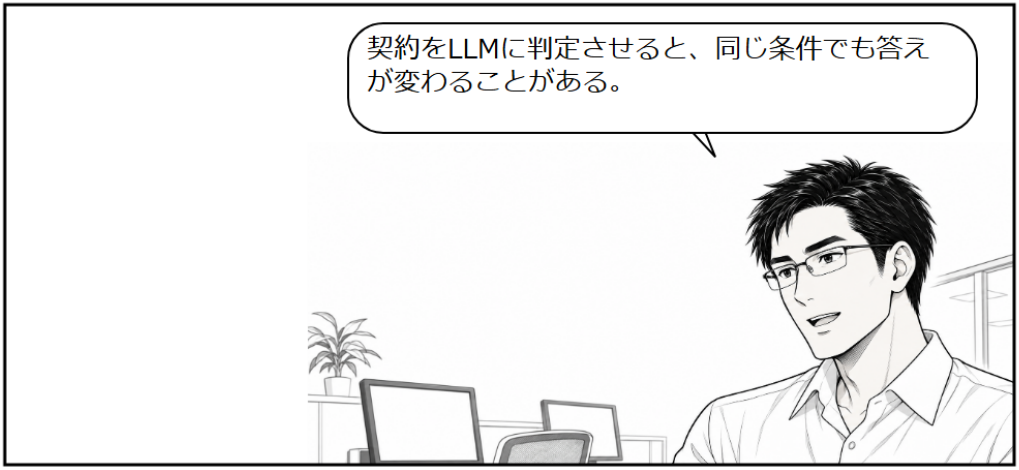
契約表：構造化

神宮寺
文書は問いと意味が似ているもの、契約は顧客情報などの条件が一致するものを探すことになるね。

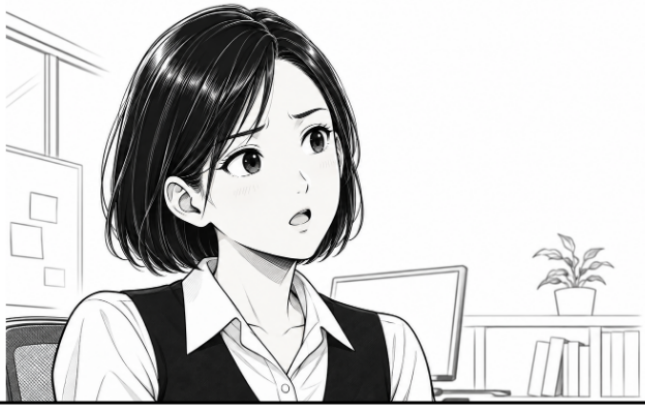
ベクトル検索とリレーショナル表の検索を、組み合わせるんですね。

ベクトル検索なら「開かない」
「固まった」「応答しない」といった
似た意味の言葉も見つけられるね。

厳密な文字の一致ではなく、
意味の近さで検索するからですね。

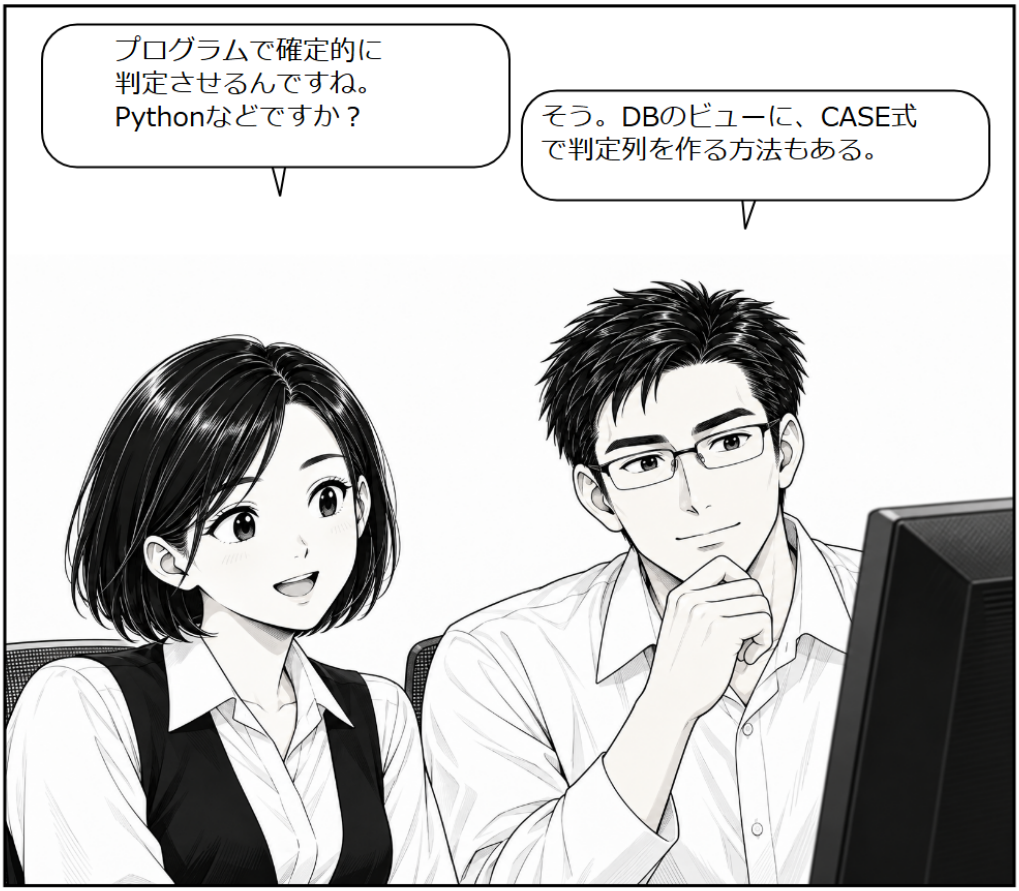

契約をLLMに判定させると、同じ条件でも答えが変わることがある。

LLMへの指示を、もっと明確にすればいいのでしょうか？

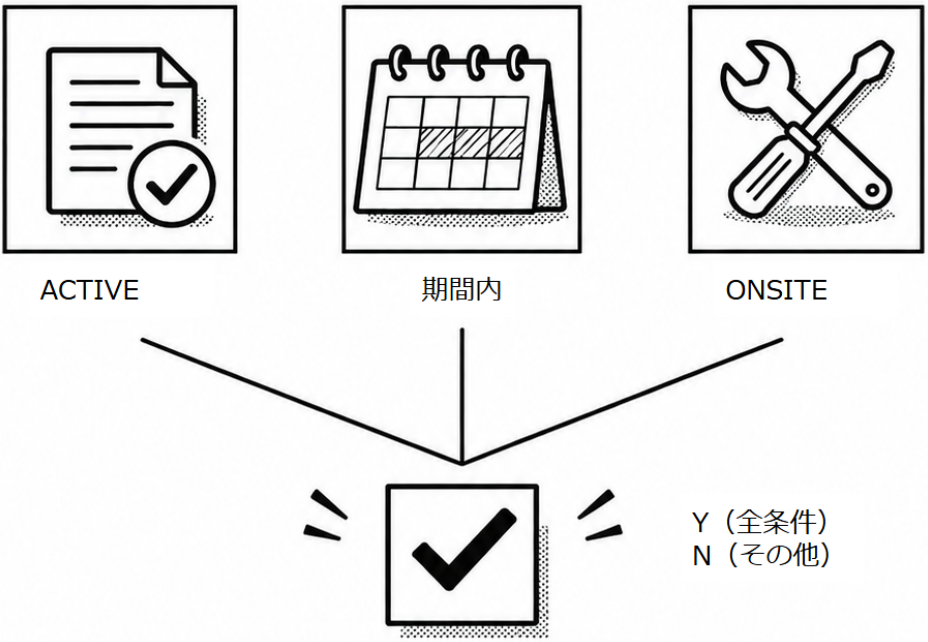


LLMは間違えることがあるし、応答にも揺らぎがある。

契約条件のように厳密さが必要な判定は、任せきりにできないんだ。


プログラムで確定的に
判定させるんですね。
Pythonなどですか？
そう。DBのビューに、CASE式
で判定列を作る方法もある。

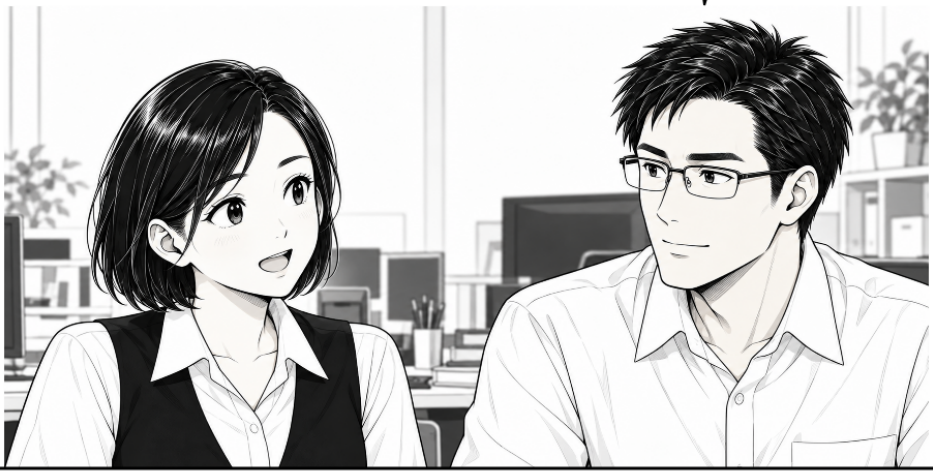
三条件を満たせばY、他はN




```
CREATE VIEW contract_judgement AS
SELECT c.*, CASE
WHEN c.status = 'ACTIVE'
AND TRUNC(CURRENT_DATE)
BETWEEN c.start_date
AND c.end_date
AND c.service_scope = 'ONSITE'
THEN 'Y' ELSE 'N'
END AS onsite_eligible
FROM contracts c;
```

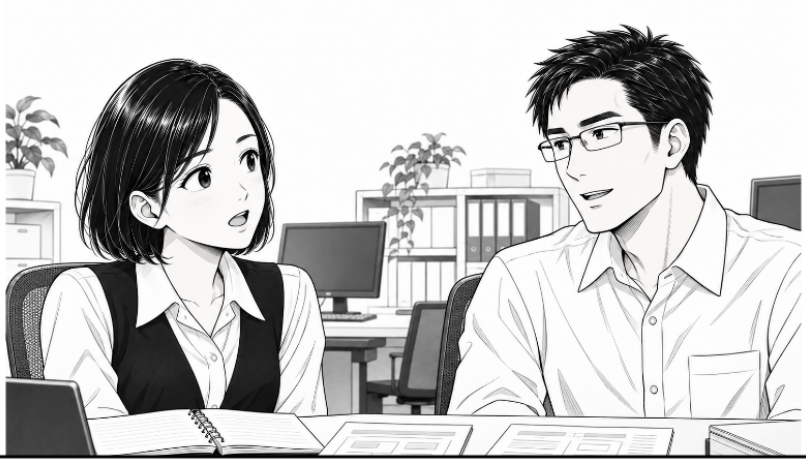
ビューにすると、どんなメリットがありますか？

ルールの適用をLLMから切り離せる。同じ契約データと日付なら、結果も同じだ。



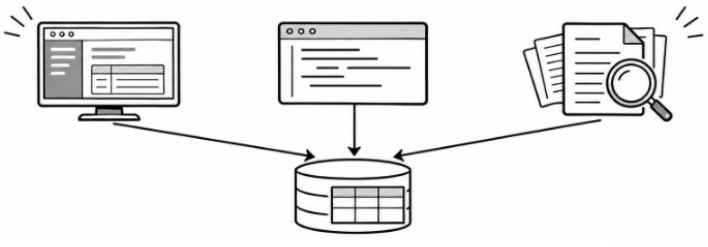
自然言語からSQLを作る場合にも使えますか？

「現地保守の対象」は判定列のY。三条件を毎回組み立て直さずに済む。

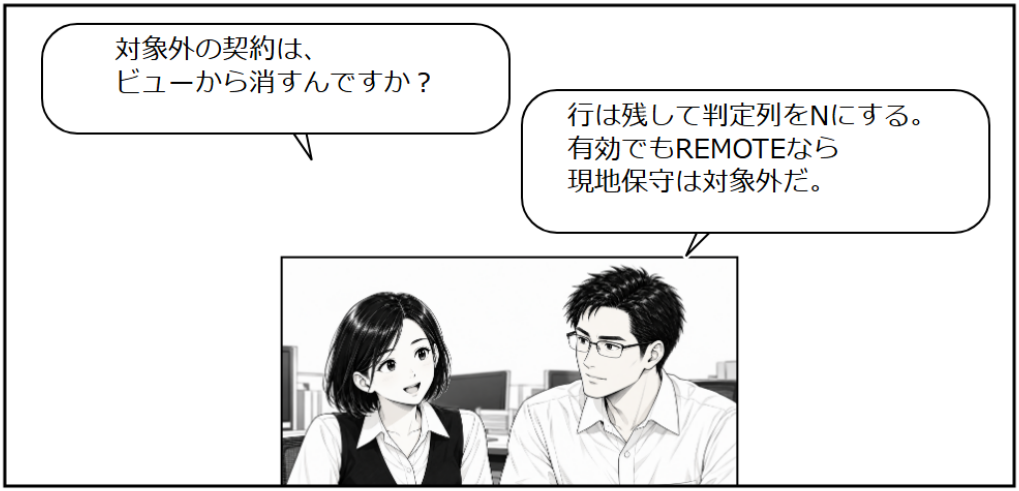


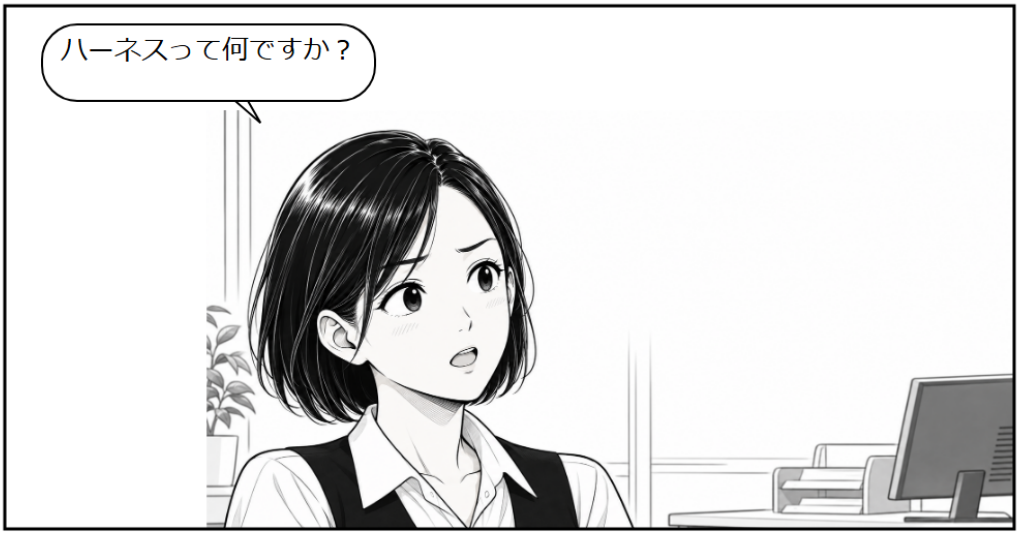
一つの業務ルールを共通で使う

画面表示　　自然言語からのSQL　　AIの契約照会処理



同じビューの判定列。呼び出す場所でルールが変わらない。





LLMと、それを支えるハーネス

LLM：意図や意味を理解する

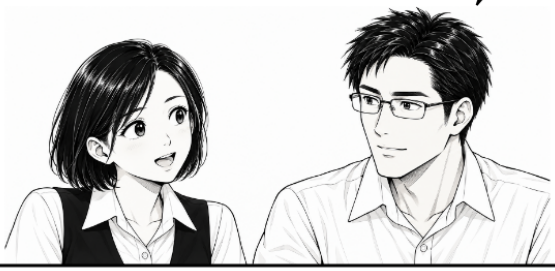
ハーネス：実行を支え、業務ルールを一貫して適用する

DBのビュー：契約の判定値を返す

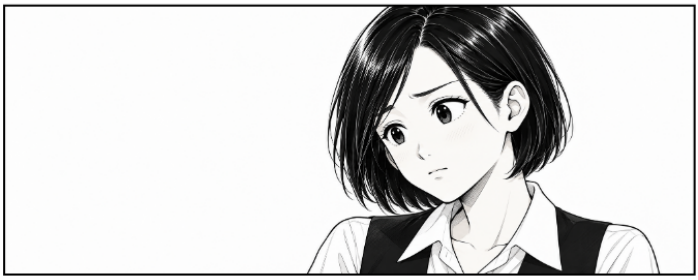
検査コード：回答や処理を判定値と照合する

意味の理解はLLMへ。
業務上の判定は切り離して、
ハーネスで守るんですね。

そう。今回の契約ルールは、DBの
ビューへまとめよう。

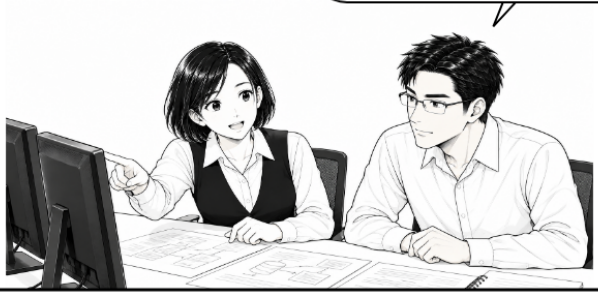


文書データと契約データは、
どこで管理したらいいのかな？

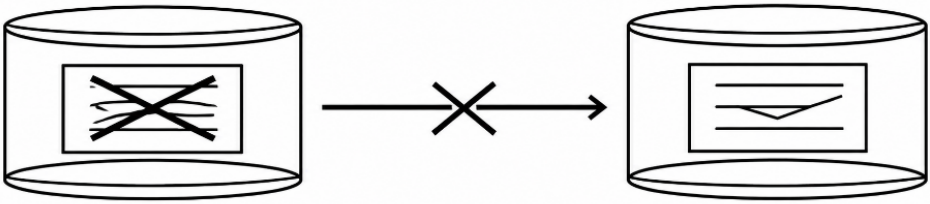


文書はベクトルDB、契約は業務
DBに置く方法は？

検索を独立して拡張できる。ただ、
変更や閲覧範囲の同期が要る。



複製の更新が遅れると

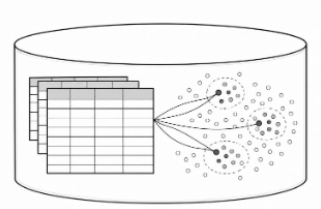


業務DB：取消済み　　検索用の複製：未反映

アイ
解約や公開取消が伝わらなければ、案内を間違えますね。

Oracle AI Databaseなら、
業務表とベクトルを同じDBで扱える。

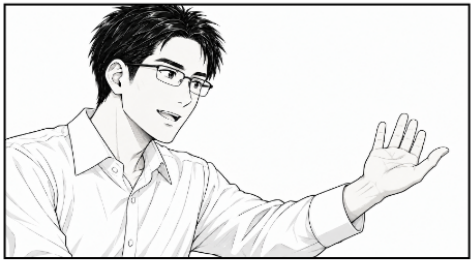
最新の業務表を直接使えるので、検索用の複製や同期の負担を減らせる。



アイ
JOINやEXISTSなら、
最新の業務表を条件にして文書を絞れるんですね。

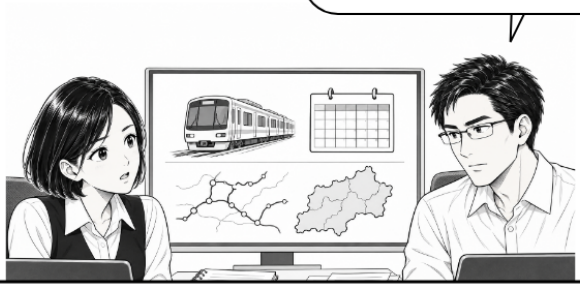
鉄道会社を想定した分析例

同じDBで扱えるのは、表と文書だけではない。
位置情報も使う鉄道の例を見よう。



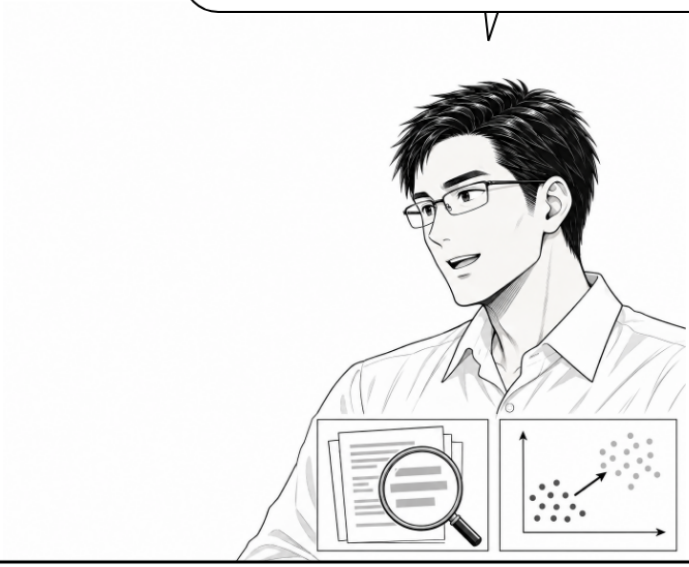
車両タイプや期間に加えて、新潟県への進入も調べられますか？

タイプと期間はリレーショナル表。
県への進入は、走行位置と県の地理空間情報で調べる。

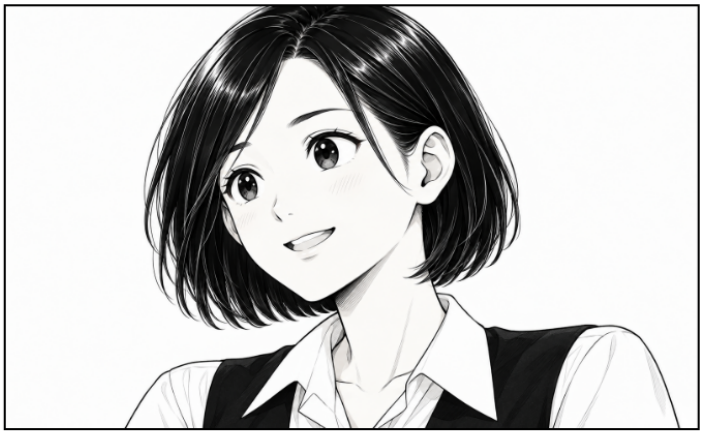


その車両・期間の報告書で、本文に「ブレーキ」か「制動機」があるかは全文検索。

その中で、凍結の影響に意味が近い報告を探すのはベクトル検索だ。



表、地図、文章の条件を、同じDBのSQLで組み合わせられるんですね。



閲覧制限や現地対応判断には、最新の契約情報を使いたいです。

最新のデータをその場で使うため、
AIをデータのそばに。
今回はOracle AI Databaseを選びます。

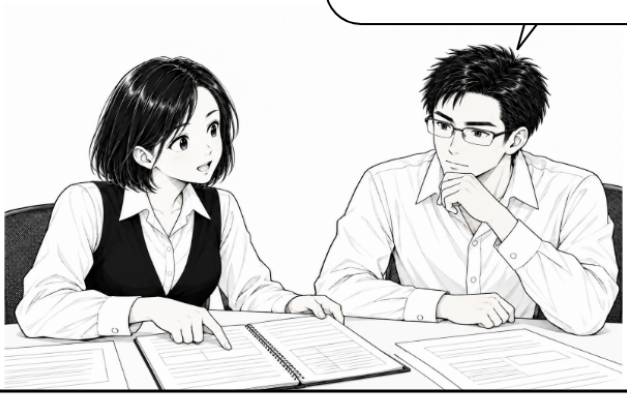
本文の変更とベクトルの更新も、同じDBの処理として設計できますね。

そうだね。次は、そのベクトルをどう作るか考えよう。

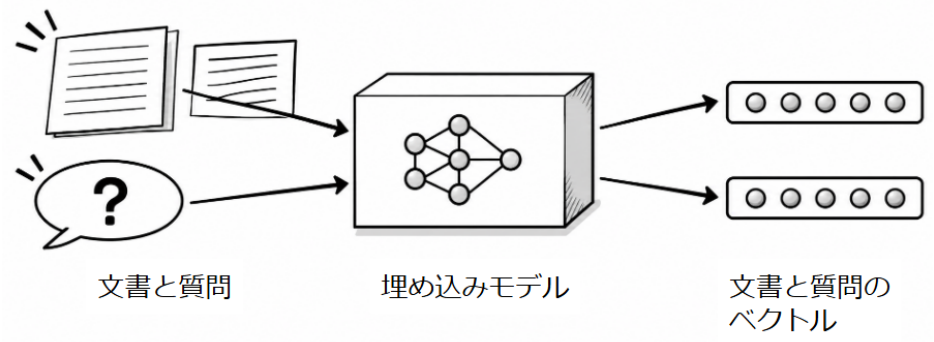
2. 埋め込みモデルをDB内へ持ち込み、DBの中でベクトルを作る

文書をベクトルにするには、
何を使うんですか？

埋め込みモデルだよ。文書と質問を
同じモデルで変換する。

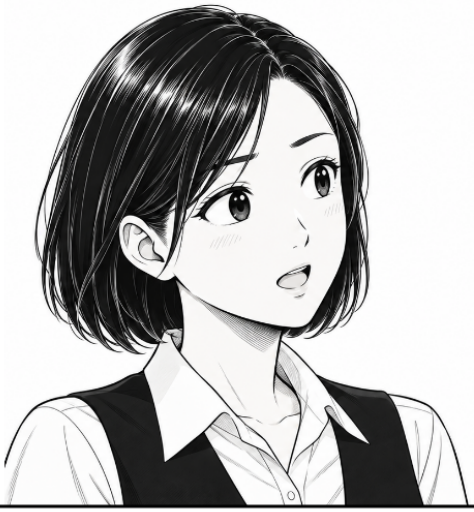


埋め込みモデルとベクトル検索

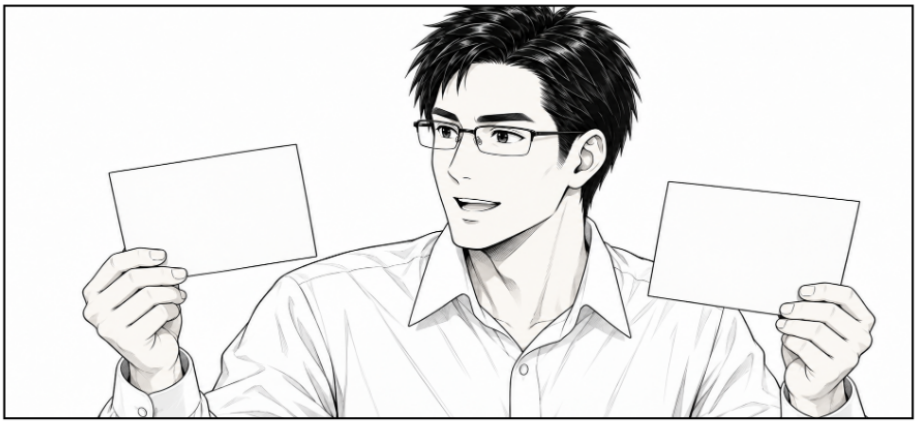


アイ
質問文のベクトルに距離が近いベクトルを持つ文書を見つけるのが
ベクトル検索ですね。

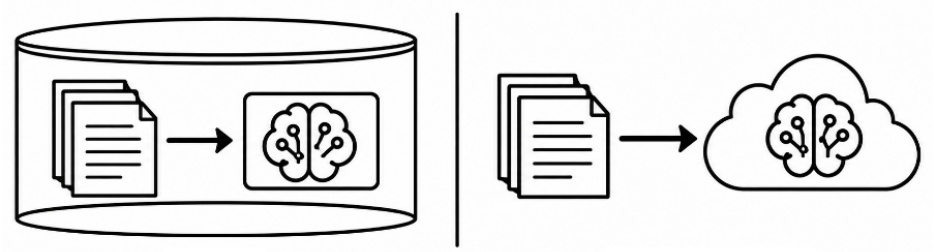
埋め込みモデルは
クラウドのAPIを使うんですか？



外部APIを使うことも、DB内に
埋め込みモデルをインポートすることもできる。



埋め込みモデルを動かす場所

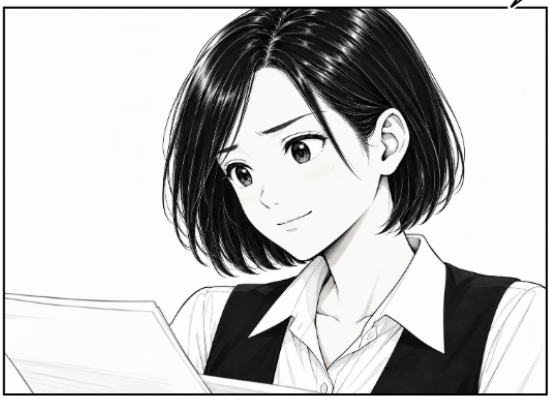


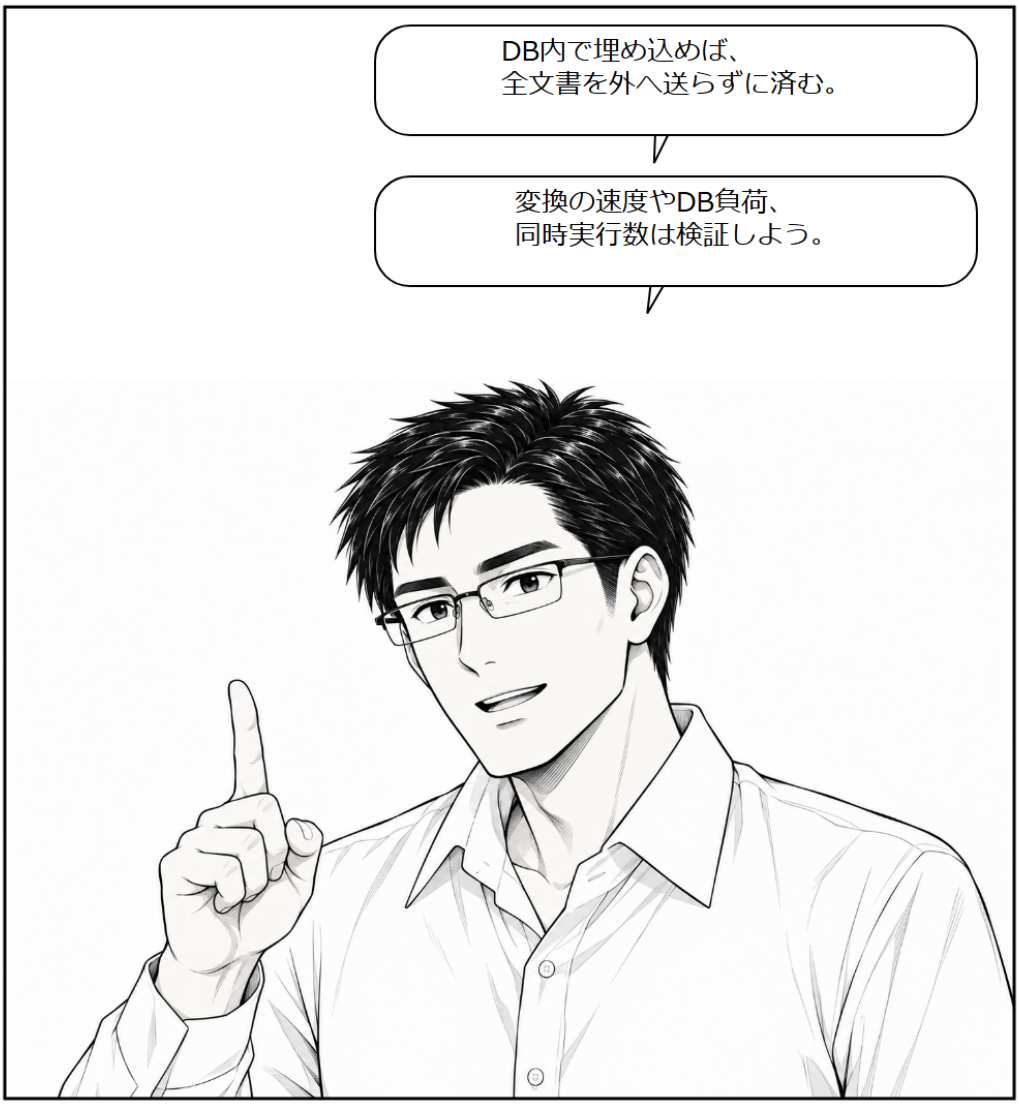
DB内

クラウドなどの
外部API

神宮寺
文書のDB登録時には、検索対象の全文書を埋め込みモデルへ送る必要がある。

回答に使う抜粋なら外のLLMへ送れます。
でも、検索準備で社内文書を全部外へ出すのは避けたいです。

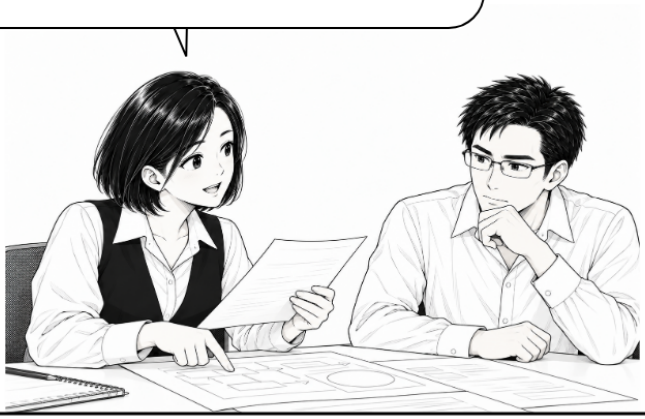



DB内で埋め込めば、
全文書を外へ送らずに済む。
変換の速度やDB負荷、
同時実行数は検証しよう。

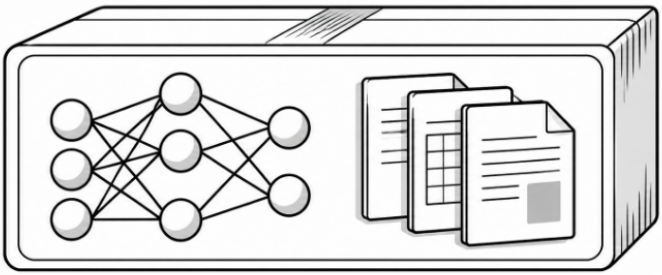
埋め込みモデルはDB内、
回答生成LLMはクラウドAPI

処理	場所	渡すデータ
ベクトルへの変換	DB内	文書と質問
回答の生成	クラウドAPI	質問と必要な文書抜粋

埋め込みはDB内、回答の生成はクラウド。役割に合わせて配置を決めます。



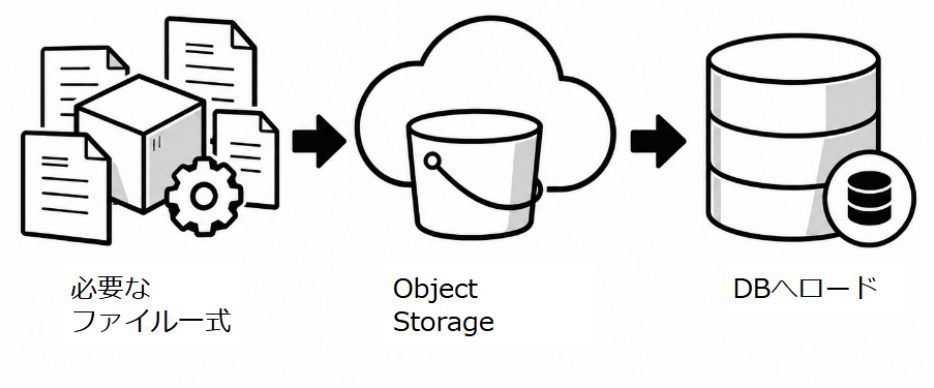
multilingual-e5-large



神宮寺
日本語対応の著名なOSS埋め込みモデル、multilingual-e5-largeを試してみてはどうかな。

神宮寺
ONNX形式へ変換してDBに取り込めば、SQLのVECTOR_EMBEDDINGで使える。

埋め込みモデルのインポート



神宮寺
認証と権限を用意して、Object Storageからモデルをロード
しよう。

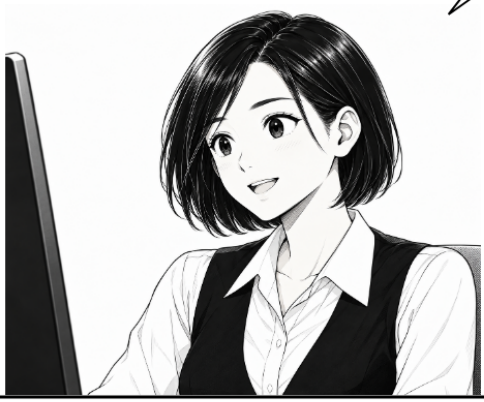
これで質問文に似ている文書の検索
は解決ですね？

では、似ている文書を見つければ
案内に使えるかな？

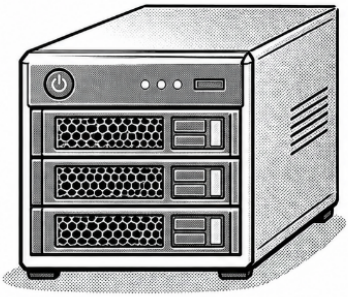
3. チケットの顧客と機種を絞ってベクトル検索する

設計の検証ケース：意味だけで検索

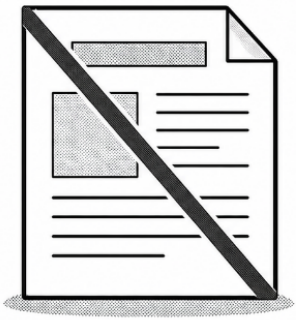
「管理画面が開かない」に近い文章なら、候補は見つかりそうです。



似ていても、そのまま使えない



別機種：BX-200



廃止済みの文書

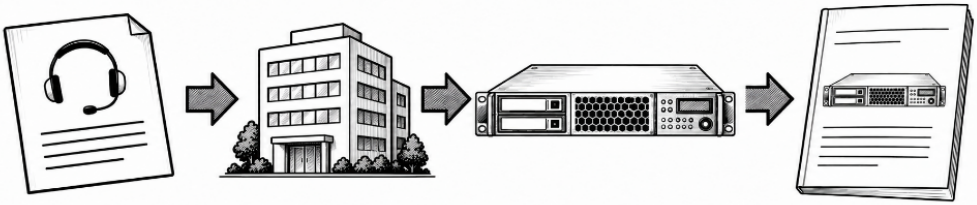
アイ
あっ、別機種。それに廃止済み。これでは案内に使えません。

別機種や廃止済みだけでなく、別顧客専用の文書も混ぜてはいけない。

同じ機種の似た症状でも、他社専用の手順を案内したら困りますね。

今回はSR001の顧客C001への回答だ。別顧客の機器や契約も混ぜない。

対応中のチケットから検索対象を決める

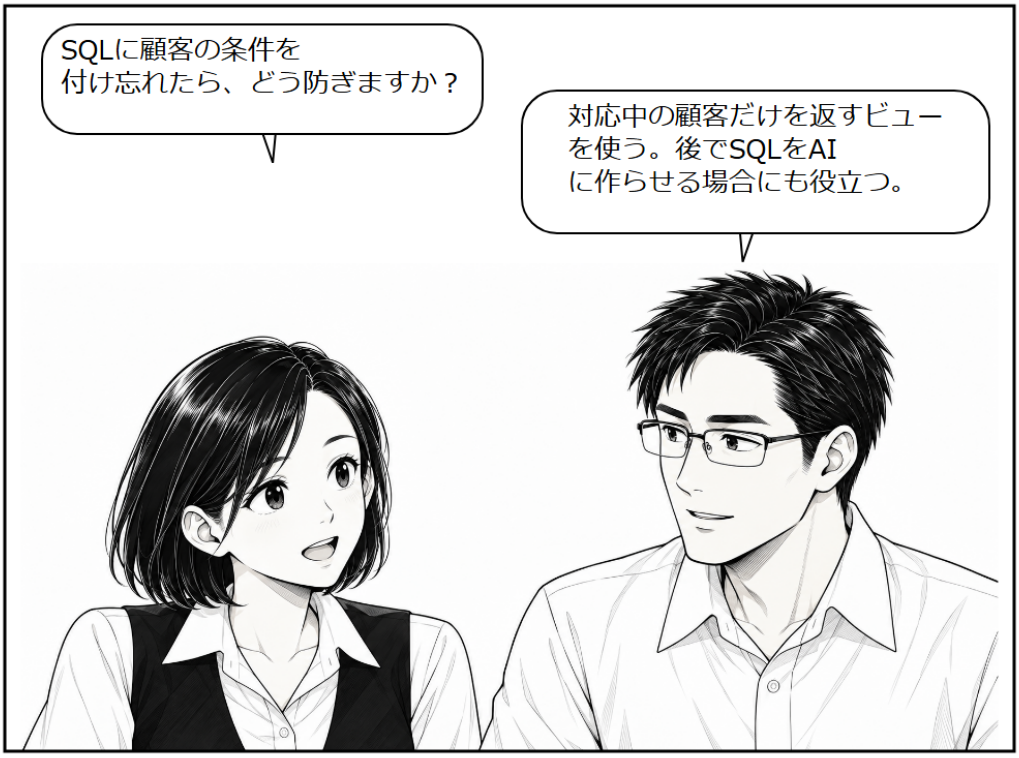


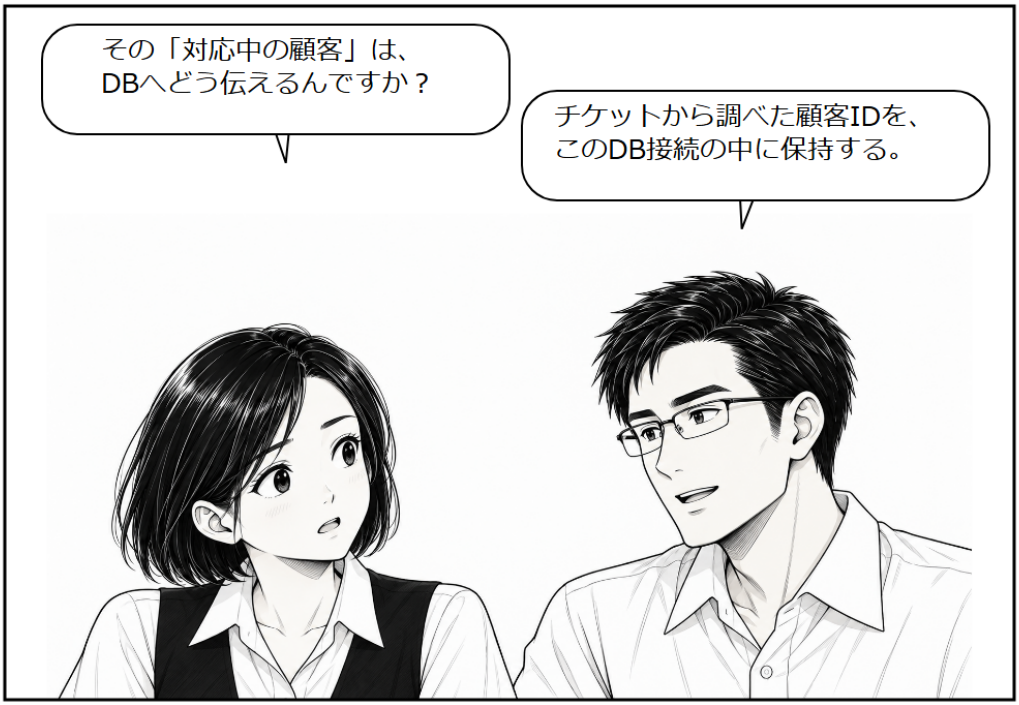
SR001：
対応中のチケット

C001：
問い合わせ元の顧客

DEV001：
その顧客の機器

AX-100：
機器の機種

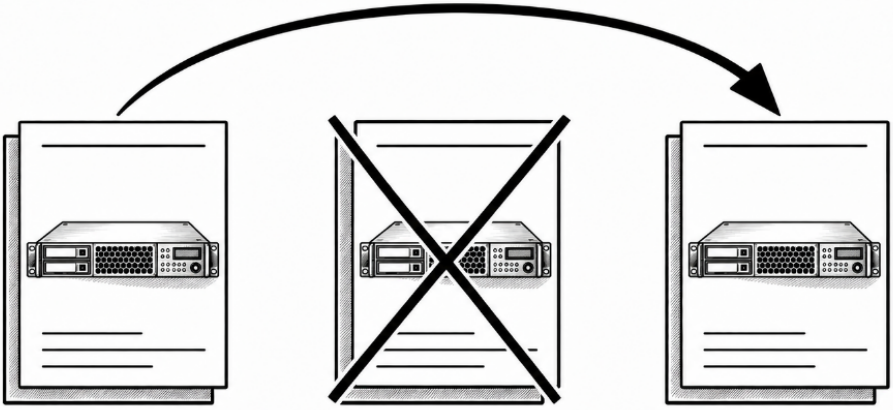

SQLに顧客の条件を
付け忘れたら、どう防ぎますか？
対応中の顧客だけを返すビュー
を使う。後でSQLをAI
に作らせる場合にも役立つ。


その「対応中の顧客」は、
DBへどう伝えるんですか？
チケットから調べた顧客IDを、
このDB接続の中に保持する。

接続の中に保持した顧客IDを、読み出せるようにしておく。

ビューのWHEREで、その値と表の顧客IDを比較するんだ。

保持した顧客IDをWHERE条件に使う




```
CREATE VIEW OJ41_VISIBLE_DEVICES
AS
SELECT d.*
FROM OJ41_DEVICES d
WHERE d.customer_id =
OJ41_CASE_CTX.customer_id();
```

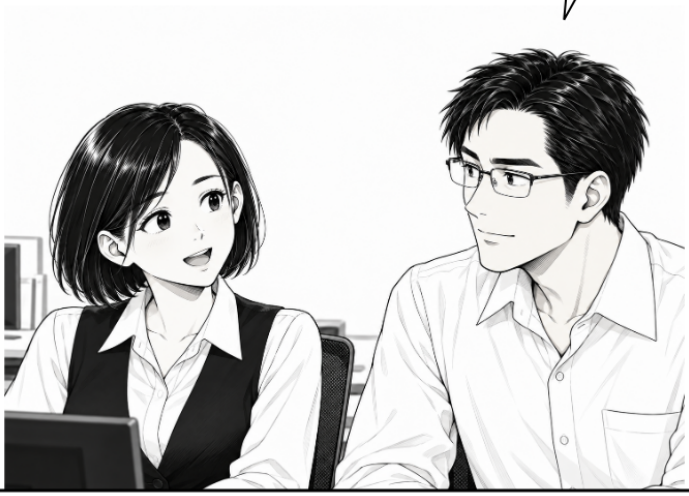
SR001を開くと、この接続にC001を保持。

customer_id()がC001を返す。

WHEREでC001の行だけを返す。

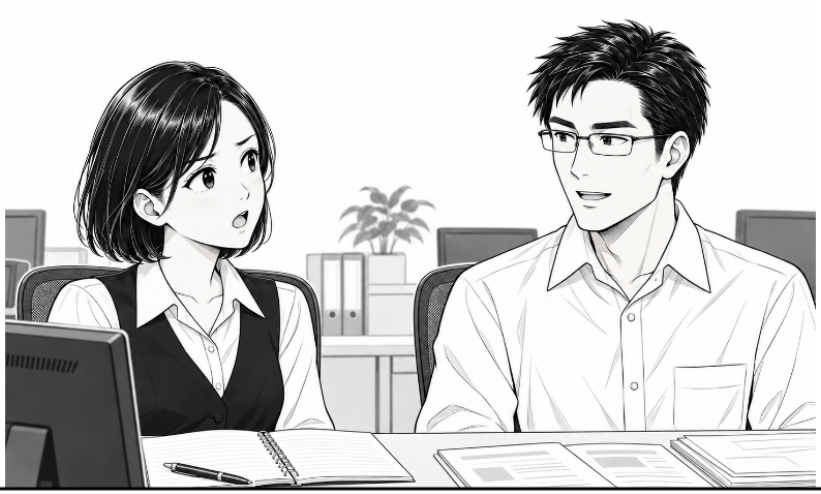
保持するだけではなく、その値をWHEREで使うから、返る行が絞られるんですね。

未設定ならNULLなので一致する行はなく、0件になる。



文書も、対応中の顧客が使えるものだけにしますか？

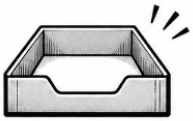
公開済みの共通文書と、
その顧客専用の文書に絞る。
検索時には機種も指定する。



文書を回答に使える範囲

公開済み：共通文書、C001専用文書

除外：C002専用、社内限定、下書き、廃止済み



チケット未設定なら0件

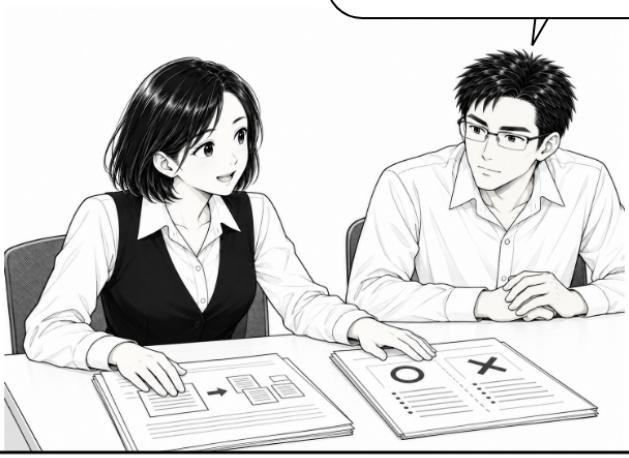
残った候補を機種で絞り、ベクトル距離で比較

ビューなら、AIが顧客条件を忘れても大丈夫ですか？

参照先とDB権限をそのビューに限定する。元の表を直接読めるままでは守れない。

契約が無効なら、文書も出しませんか？

許可された技術文書は案内できる。
現地保守の可否とは別だよ。

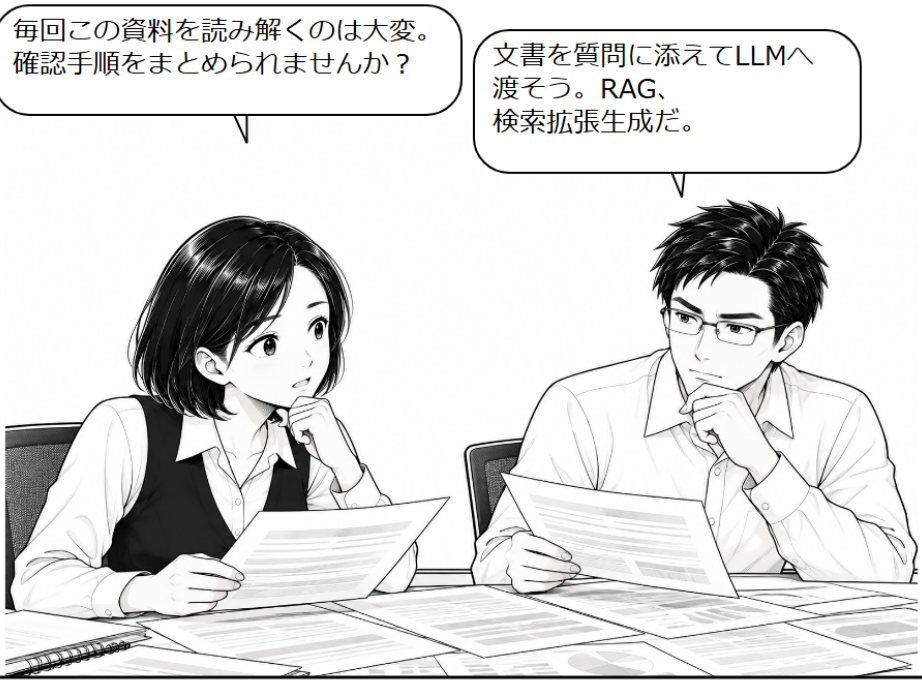


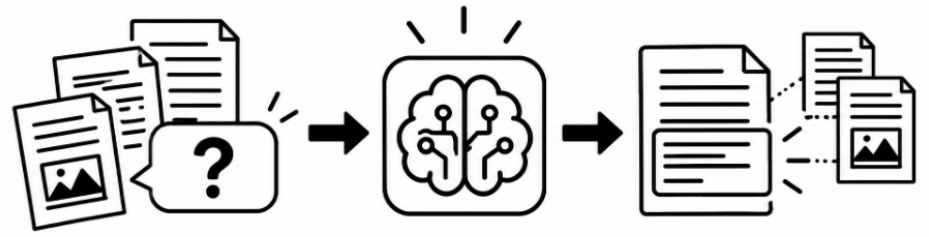
担当者の認可、別顧客や未設定のケースも確認します。

接続に残した顧客情報は、
成功・失敗のどちらでも消去します。

文書を探す設計はできました。次は、
担当者が使える回答にしたいです。

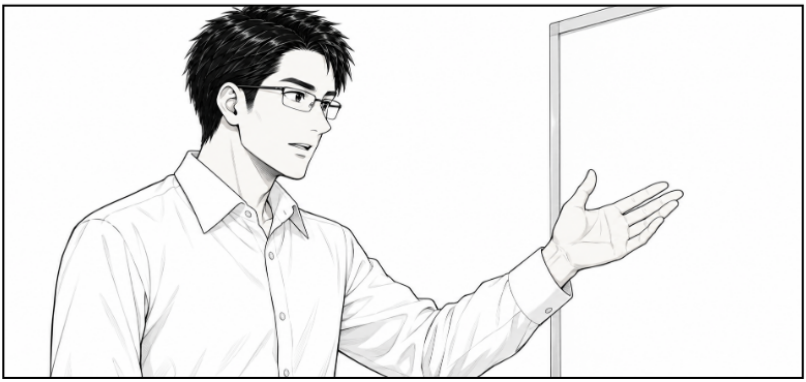
4. 検索した文書を根拠にRAG（検索拡張生成）で回答する





アイ
案内できる文書を、回答の根拠にするんですね。

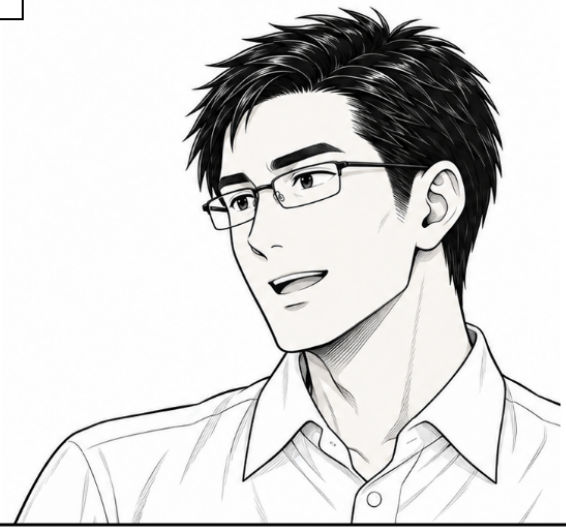
このRAGは、DB内のPL/SQLで
検索・生成・検査の順を固定する。
どの処理をするかはAIに選ばせない。



回答例：管理画面が開かないとき

電源・LAN・疎通を確認し、
ログを保存します。
再起動の前に影響を
確認してください。
（根拠：KB001）

再起動前の注意も
書けているね。

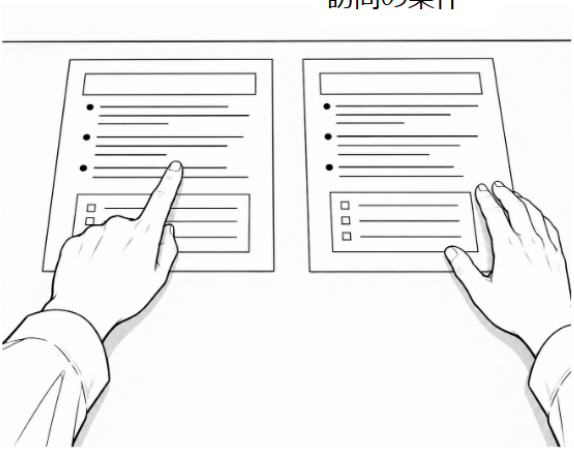


根拠の照合

KB001：
確認手順

KB012：
連絡と
訪問の条件

アイ
取得本文と、回答の
文書ID・引用を
照合します。

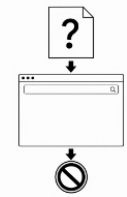


引用が原文にあっても、回答の意味まで正しいとは限りませんね。

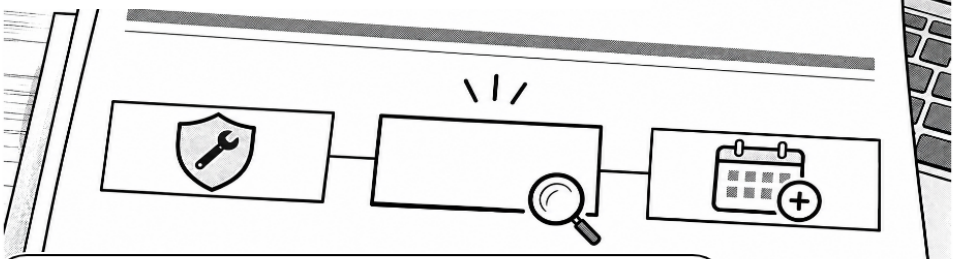
そう。無関係な引用や、
本文を超えた断定を確かめよう。
根拠がない場合もね。

根拠不足①：候補がない

「E999のエラーには？」と聞かれた。
E999を含む文書に絞ると0件。
LLMを呼ばず、根拠不足を返す。

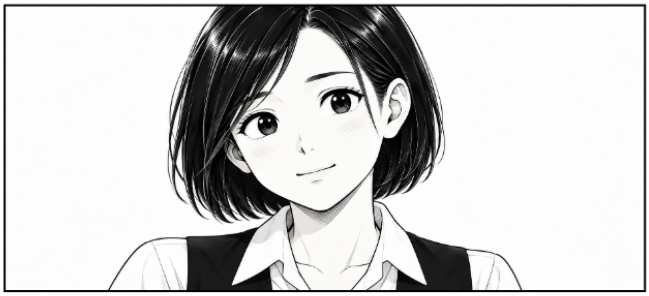


根拠不足②：文書はあるが、保証延長の金額がない

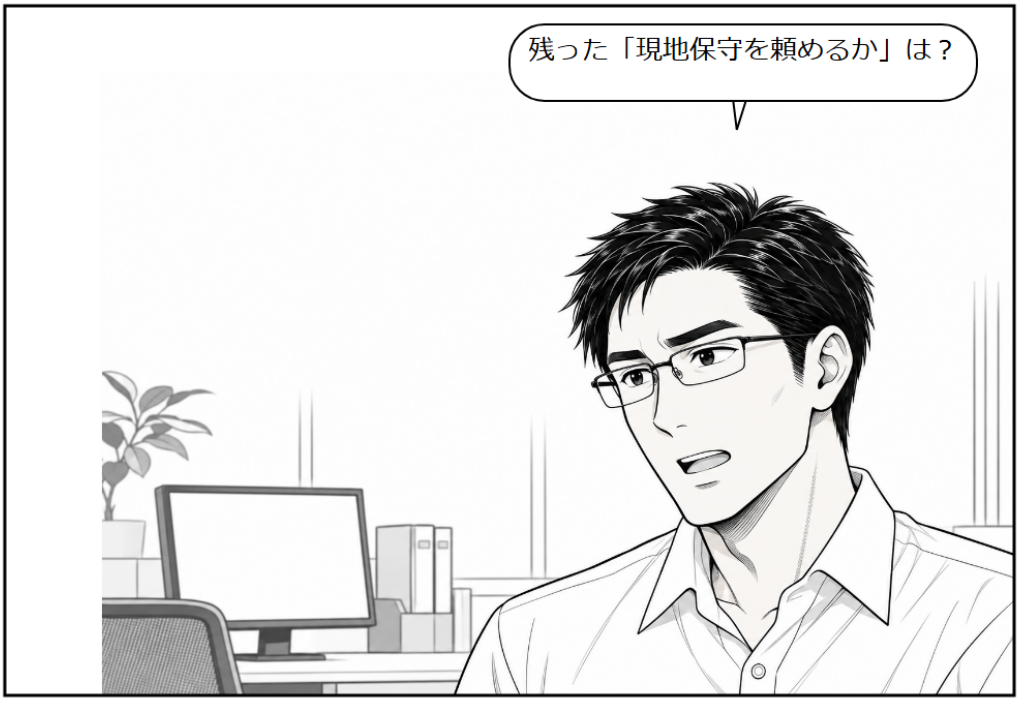


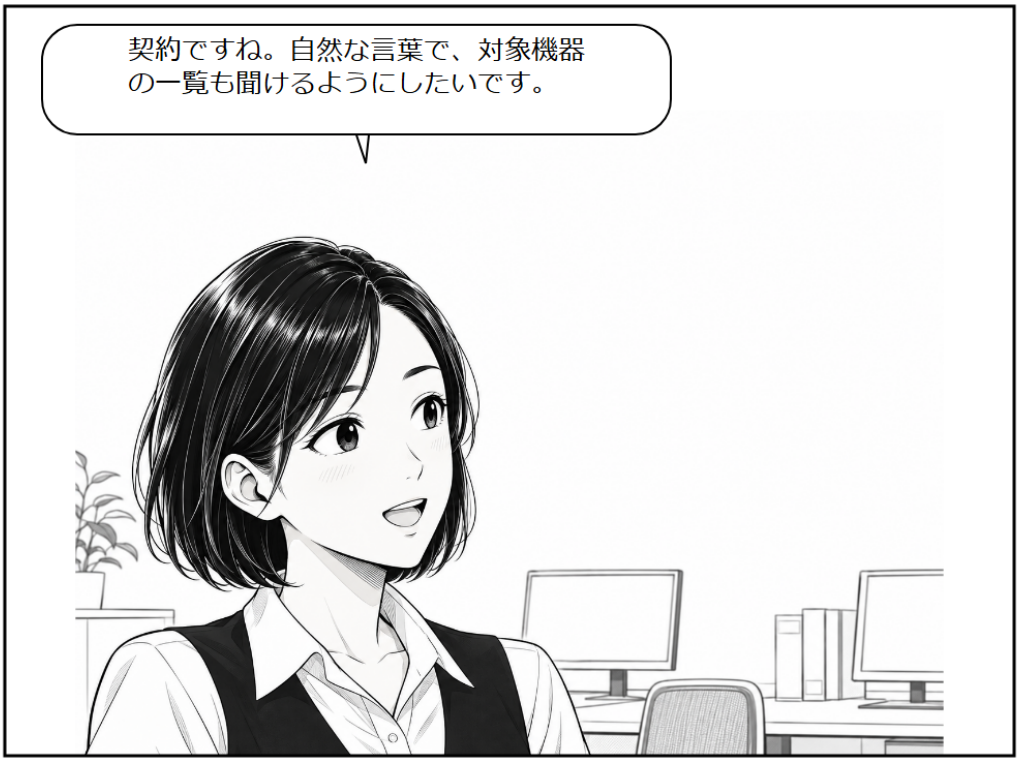
神宮寺
検索できても、質問への答えが載っているとは限らない。

分からない答えは作らない。候補ゼロと、答えの記載なしを別々に試します。



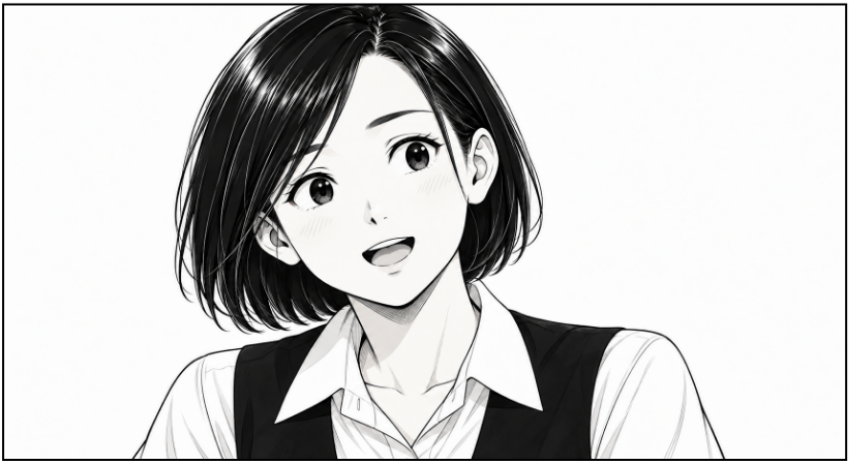
固定順のRAGを採用します。検索の成否と、回答が根拠に沿うかを分けて評価します。


残った「現地保守を頼めるか」は？


契約ですね。自然な言葉で、対象機器
の一覧も聞けるようにしたいです。

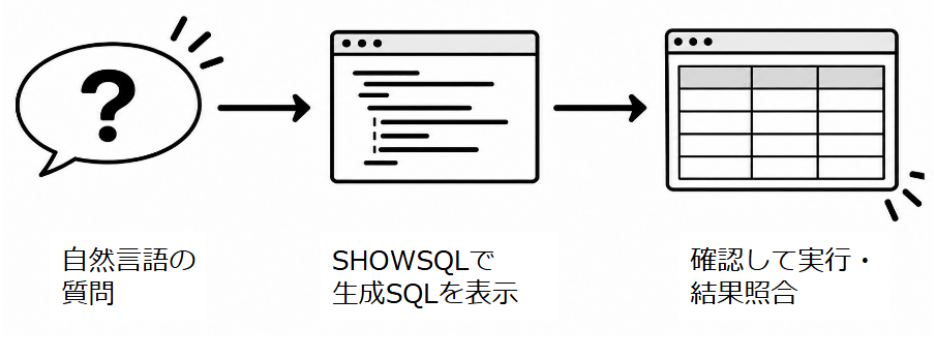
5. NL2SQL で契約情報を照会する

「この顧客の現地保守対象機器を一覧にして」のような質問ができるようにしたいです。
NL2SQLの出番ですか？



Oracleには、自然言語からSQLを作る
Select AIがある。これを使って検討し
てみよう。

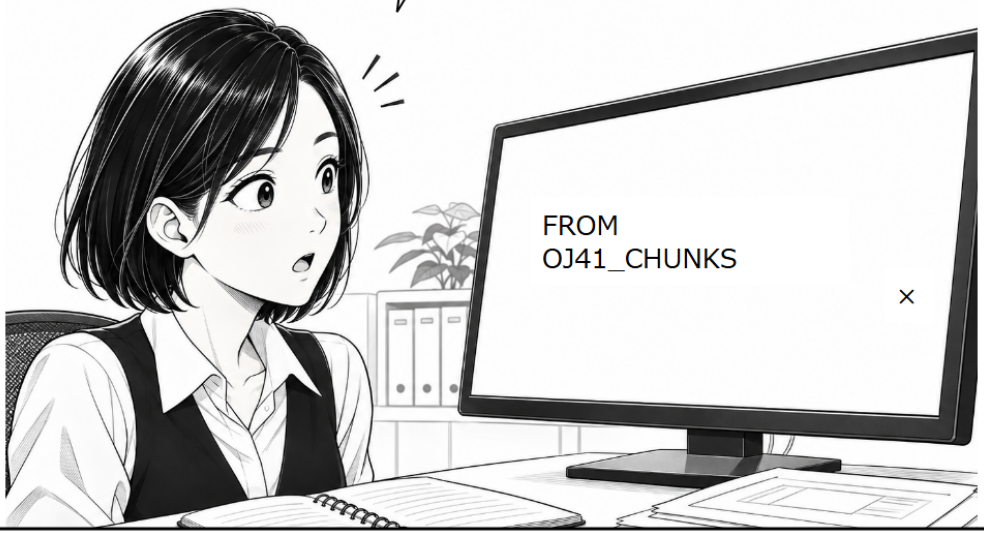
アイの試作テスト



神宮寺
まずSQLを見る。その後、実行した結果を正解と比べよう。

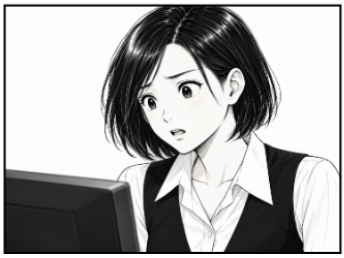
参照先を制限する前の試作

あれ、違う表を検索しています。名前だけでは、表の意味が伝わらないんですね。



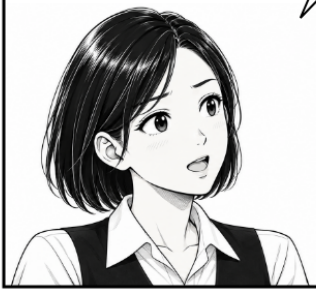
顧客のWHERE条件も誤り

表を直して再試行したら、
今度は別の顧客の機器が混ざりました。
WHERE句の条件が違います。



Select AIには、参照先の
制限や注釈など、生成を改善する手段
がある。ビューと組み合わせよう。

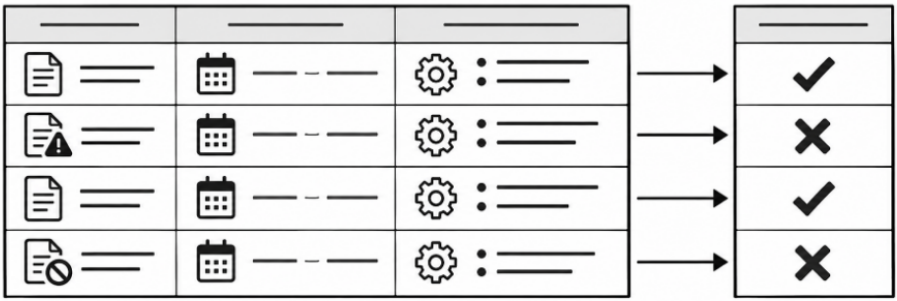
許可したビューだけを使う設定にします。
列の意味も伝えられますか？



列の意味は注釈で伝えられる。直したSQLを例にするFeedbackも使える。

現地保守の可否は、先ほどのビューの判定列を使えますね。
そう。三条件をLLMに考え直させず、判定済みの列を使おう。

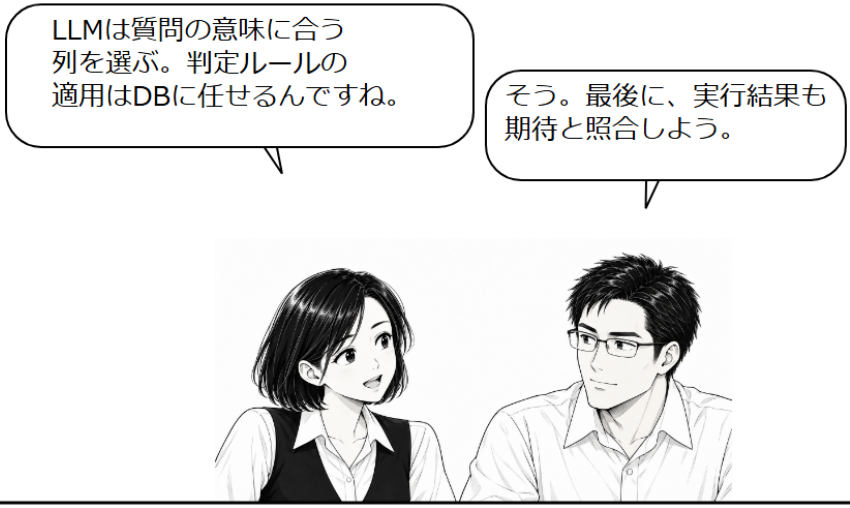
改善後の生成SQL：判定列を使っている



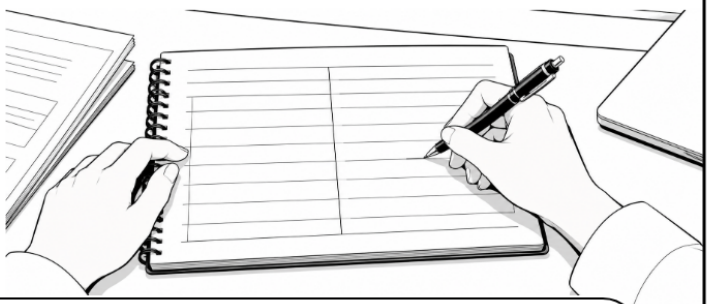
```
SELECT device_id
FROM OJ41_VISIBLE_CONTRACTS
WHERE onsite_eligible = 'Y';
```

質問：「この顧客の現地保守対象機器を一覧にして」

このビューは顧客の範囲と、CASEの判定列をまとめたもの。



期待する結果と照合



アイ
対象顧客だけになりました。期限切れと現地保守なしも対象外。今回のテストは期待どおりです。

注釈はSQL生成を助けるもの。参照範囲を守るのは、ビューとDB権限だよ。

自然な言葉の質問と、
DBで守る条件。
役割を分けて設計できました。

では、手順と契約を
一度に聞かれたら？

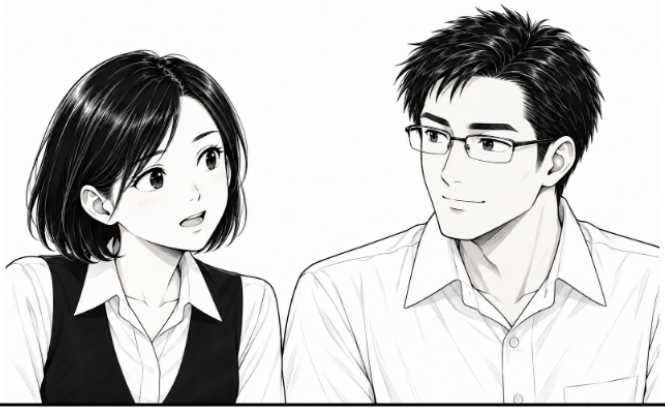
6. 文書検索と契約照会をAgentのツールにする

手順だけ、契約だけ、両方。依頼に合わせて処理を選びたいです。

Agentなら、目的に応じてLLMが道具を選び、結果を見て次の処理を考える。

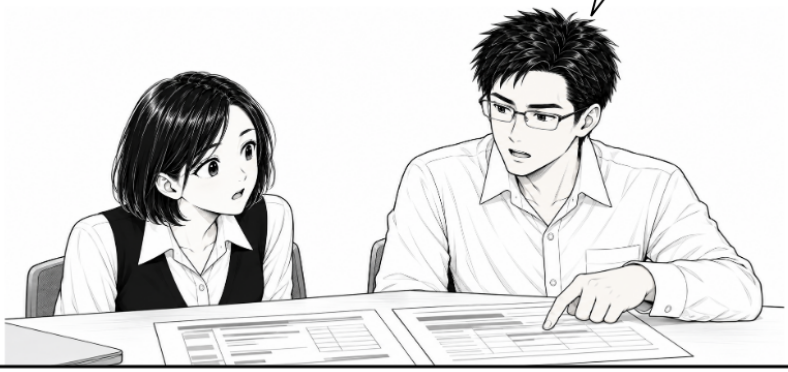
道具って、システムに用意する処理のことですか？

そう。Agentが呼び出せる形にした処理を、ツールと呼ぶ。

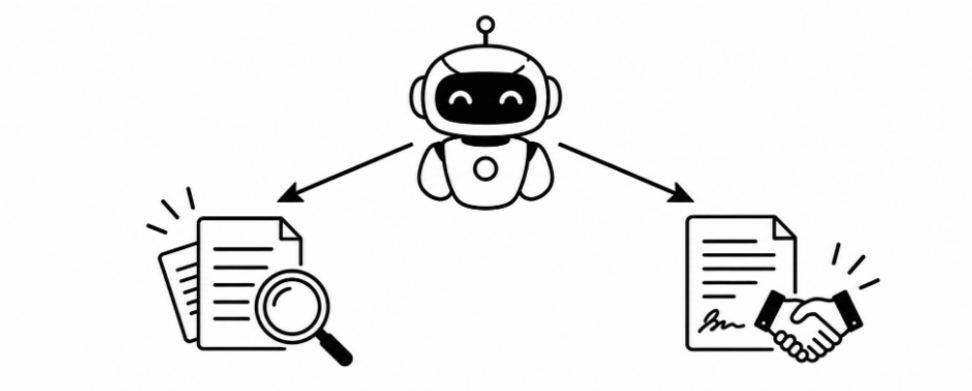


NL2SQLとの関係も整理したいです。

ここではSQLは固定する。
Agentがツールを選び、検索語などの引数を渡す仕組みを見よう。



固定SQLを実行する二つのツール



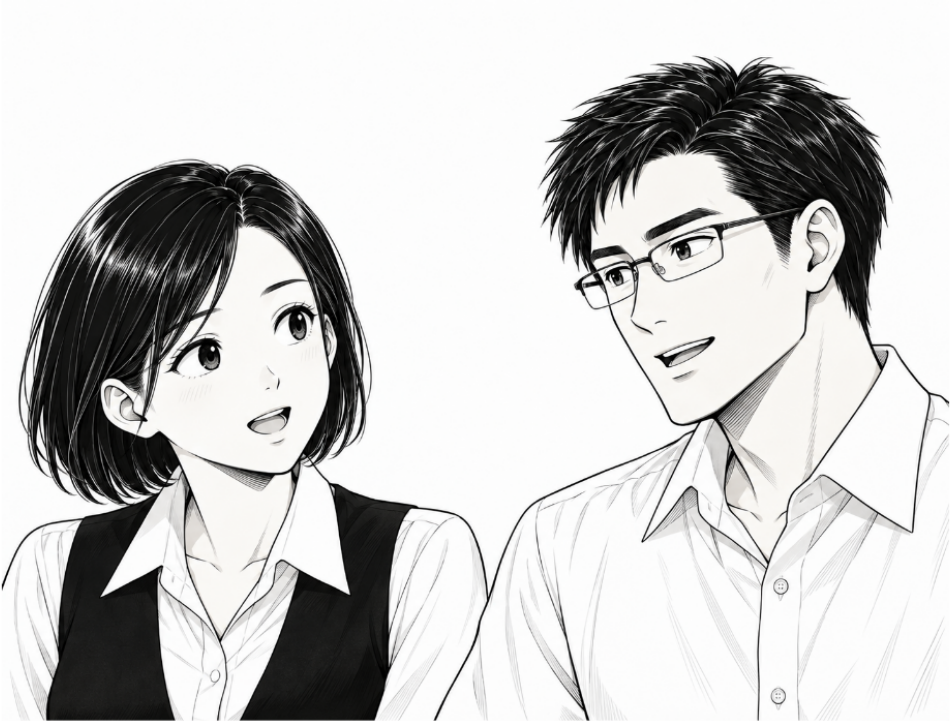
文書検索：条件で絞って距離順

契約照会：ビューの判定列を読む

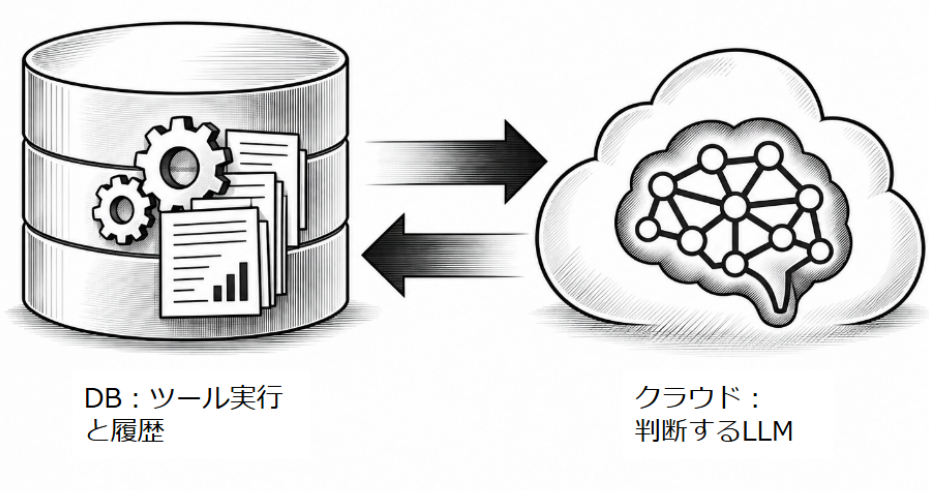
神宮寺
文書検索も契約照会も、中身は決まったSQL。
Agentは呼ぶ道具と、
検索語などの引数を選ぶ。

このAgentを動かす基盤は、
どこに置くんですか？

Select AI Agentを使おう。
DB内でツールの実行や履歴を扱い、
LLMとはAPIでやり取りする。



Select AI Agentの実行基盤



アイ
基盤はDBの中。LLMへ渡す指示・ツール定義・結果の内容も設計します。

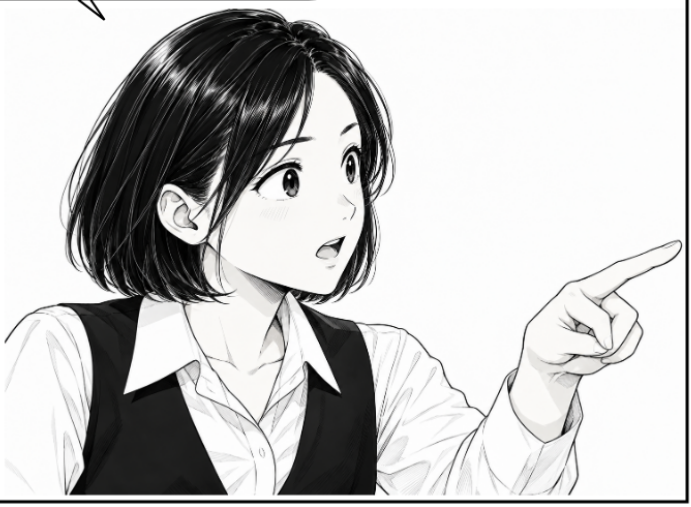
失敗例①：ツールを呼ばずに回答

失敗例②：実行エラーを成功扱い

呼んでいても安心できない。実行に失敗したのに、成功したように答えることもある。

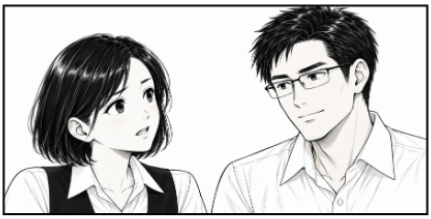
失敗例③：呼び出しの引数違い

DEV001の依頼なのに、DEV005を渡していました。結果の機器も違います。



LLMに「実行を確認して」と、
もう一度頼めばいいですか？

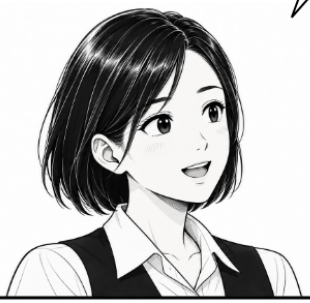
その確認も間違えることがある。
ここもハーネスの仕事だよ。



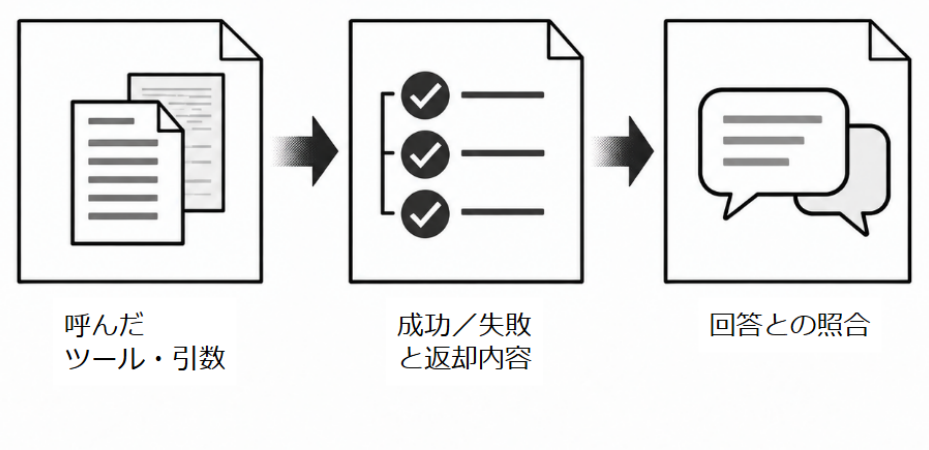
今のチケットをコードで保証する

担当権限を確認した今のチケットから、対象機器をアプリで設定する。不一致はDBで止める。

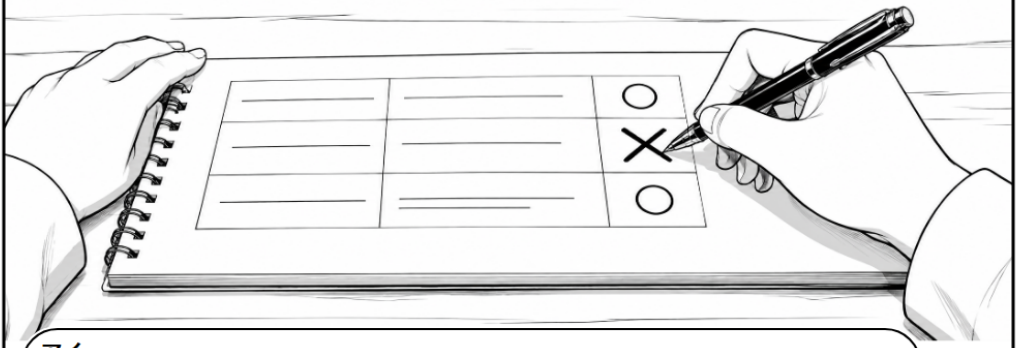
検索語はLLMに任せても、チケット
と対象は勝手に変えさせないんですね。



ツールの履歴で実行を確かめる



神宮寺
Select AI Agentにはツールの呼出履歴を参照するビューがある。記録と返却内容をコードで確かめよう。

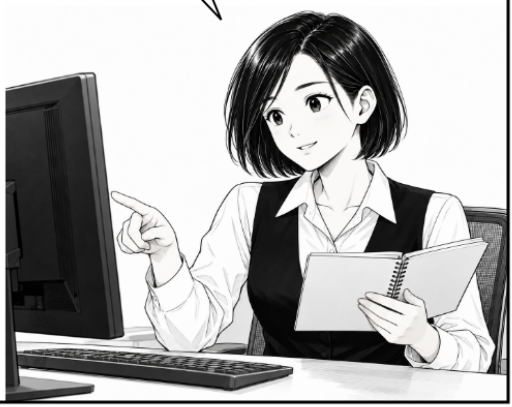


アイ
呼出しなし・実行失敗・対象違いなら、成功した回答として通さない。引用や契約値を照合する検査も用意しました。

検査を加えて、もう一度

KB001：電源・LAN・疎通の確認とログ保存

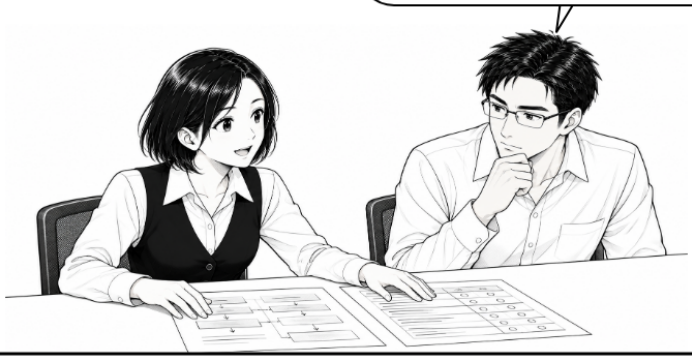
二つのツールを実行し、
対象も結果も一致しました。
再起動前の影響確認も、回答に残っています。



契約判定と訪問承認を分ける

現地保守の対象なら、担当者の承認を確認。対象外や不明なら、承認へ進めません。

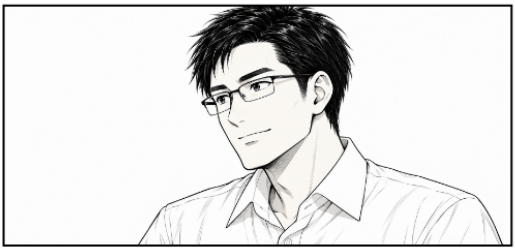
対象・承認済み・予約済みは別だね。文書の案内も、許可された範囲で続けよう。



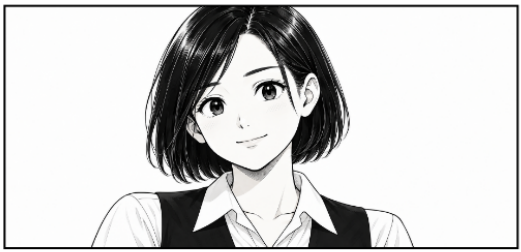
LLMとハーネスの役割分担

意味の理解はLLM、
業務ルールの適用はハーネス。
任せる範囲が見えてきました。

データのある場所でAIの検索や処理を動かす。
それがインデータベースAIだよ。
ハーネスも組み合わせて業務の条件を守ろう。

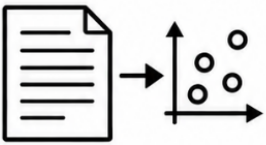


はい。使えるデータ、引数、失敗時の扱いまで含めて。Agentアプリを設計できそうです！

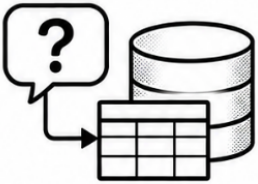


データのそばに、AIを。

Oracle AI Database :
表・文書・ベクトル・位置情報を
一つのDBで



DB内の埋め込みモデル :
文書を外へ送らずベクトル化



Select AI :
自然言語からSQL。
ビューへの権限制限と注釈を組み合わせる

Select AI Agent :
DBでツールを実行し、履歴を確認。
LLMとはAPI連携

ハーネス :
ビュー・PL/SQL・検査で
業務ルールと範囲を守る